

Employee Scheduling & Time Tracking App

Availability, shift assignment, acceptance workflow, time clock, and roster generation for hourly staff.

Prepared: August 24, 2026

Contents: Overview · Roles · Features · Business Rules · Data Model · User Flows · Wireframes · Tech Approach · Open Questions · Build Phases

1. Overview

A two-sided web app for scheduling hourly/shift staff:

- **Employees** submit their availability up to two weeks ahead, view and respond to shifts they're offered, clock in/out, and track their hours.
- **Managers** see which shifts are understaffed, assign employees from the pool who are available, and get a confirmed roster once every assigned employee has accepted.

Two roles, one shared shift/availability engine underneath.

2. User Roles

Role	Can do
Employee	Set availability, view assigned shifts, accept/reject offers, clock in/out, view own timesheet
Manager / Employer	View shift staffing board, assign employees to shifts, view roster once confirmed

Not asked for, but worth flagging as a likely future role: a Super Admin who manages locations, shift templates, and manager accounts across multiple sites — see Section 10.

3. Feature List

3.1 Employee Portal

1. **Login** (email/password to start; SSO can be added later)
2. **Set Availability** — rolling 14-day window from "today." Employee marks each day available/unavailable and, if available, sets a from/to time range.
3. **View Assigned Shifts** — date, day, time, location, and status (pending / accepted / rejected).
4. **Accept / Reject** a shift offer.
5. **Clock In / Clock Out** — one-tap button, current status shown clearly.
6. **Personal Dashboard** — hours worked today/this week, days worked, shift history.

3.2 Manager Portal

1. **Login**
2. **Shift Staffing Board** — every shift for the week, how many people are staffed vs. the 3-5 requirement, and a status (Needs Staff / Pending Acceptance / Fully Staffed / Unfilled).
3. **Assign Employees to a Shift** — click a shift, see everyone who marked themselves available for that slot, assign up to 5.
4. **Auto-Reassignment** — if an assigned employee rejects, the system automatically offers the open spot to the next available, unassigned employee for that slot.
5. **Weekly Roster** — once a shift's assigned employees have all accepted (and the 3-person minimum is met), it locks in and appears on the published roster.

4. Key Business Rules

These are the rules implied by the brief — flagged clearly so you and your developer agree on them before build:

Availability window: rolls forward daily. If today is Aug 24, the employee can enter availability through Sep 7 (14 days out). Each day, the window advances by one day.

Shift capacity: minimum 3, maximum 5 employees per shift.

Assignment is manager-initiated: the manager picks who to offer a shift to, from the list of employees who marked themselves available for that time slot. It's not fully automatic on the first pass.

Rejection → auto-reassignment: if an employee rejects, the system automatically offers the same slot to the next available (not-yet-offered) employee, and repeats until the slot is filled or the pool of available employees is exhausted.

Roster confirmation: a shift moves onto the published roster once every currently-assigned employee for it has accepted **and** the minimum of 3 is met. If the pool runs out before reaching 3, the shift is flagged **Unfilled** for the manager to handle manually, rather than silently blocking the rest of the roster.

5. Data Model (suggested)

Entity	Key fields
User	id, name, email, password_hash, role (employee/manager), location_id
Location	id, name, address
Availability	id, employee_id, date, start_time, end_time, is_available (bool)
Shift	id, location_id, date, start_time, end_time, min_staff (default 3), max_staff (default 5), status (open / partially_staffed / pending_acceptance / confirmed / unfilled)
ShiftAssignment	id, shift_id, employee_id, status (offered / accepted / rejected), offered_at, responded_at
TimeEntry	id, employee_id, shift_id (nullable), clock_in, clock_out
Roster	derived view: all Shifts with status = confirmed for a given week, grouped by location

`ShiftAssignment` is the table that drives the auto-reassignment logic — when a row's status flips to `rejected`, a background job (or synchronous call) finds the next eligible employee from `Availability` who doesn't already have an `offered` / `rejected` row for that shift, and creates a new `offered` row for them.

6. Core User Flows

A. Employee sets availability

- Employee logs in → lands on Availability screen showing the next 14 days.
- Toggles each day available/unavailable, sets time range if available.
- Saves. Availability becomes visible to managers immediately.

B. Manager assigns a shift

- Manager opens the Shift Staffing Board for the week.
- Clicks a shift row that needs staff.
- Sees a list of employees who marked availability covering that slot, filtered to exclude anyone already offered/rejected for it.
- Clicks "Assign" next to a name → creates a `ShiftAssignment` with status `offered`.
- Employee gets notified (in-app at minimum; email/SMS optional, see Section 9).

C. Employee responds to an offer

- Employee sees the shift as "Pending" on their dashboard.
- Accepts → status `accepted`.
- Rejects → status `rejected`, triggers auto-reassignment (flow D).

D. Auto-reassignment

- On rejection, system looks at the remaining eligible, available employees for that shift.
- Automatically creates a new `offered` assignment for the next one in the queue (ordering needs a decision — see Section 9).
- Repeats on each rejection until the slot count is filled or nobody's left to offer it to (→ shift flagged `unfilled`).

E. Roster confirmation

- System checks each shift: are all current assignments `accepted`, and is the count ≥ 3 ?
- If yes, shift status → `confirmed` and it appears on the manager's Weekly Roster view.
- If a week has some shifts still pending/unfilled, the roster view shows those clearly rather than hiding them.

F. Clock in / clock out

- Employee taps "Start Clock" → creates a `TimeEntry` with `clock_in` = now.
- Taps "End Clock" → sets `clock_out` = now.
- Hours = `clock_out - clock_in`, rolled up into daily/weekly totals on the dashboard and timesheet.

7. Wireframes

Low-fidelity wireframes for both sides of the app — meant to pin down layout and flow, not final visual design.

7.1 Employee — Set Availability

ShiftApp

EMPLOYEE

Set Your Availability

Rolling 14-day window — Mon, Aug 24 to Mon, Sep 7. Updates each day as the window moves forward.

DATE	AVAILABLE?	FROM	TO	NOTES
Mon, Aug 24	☒	08:00 AM	04:00 PM	
Tue, Aug 25	☒	09:00 AM	05:00 PM	
Wed, Aug 26	☐ Not available			
Thu, Aug 27	☒	08:00 AM	02:00 PM	

(remaining days through Sep 7 continue in the same list)

☒ Available (toggle on, set a time range)

☐ Not available (toggle off)

Unsaved changes are highlighted until you save.

Save Availability

The 14-day availability entry screen: toggle each day on/off and set a time range when available.



Hi, Sam — here's your day

TIME CLOCK

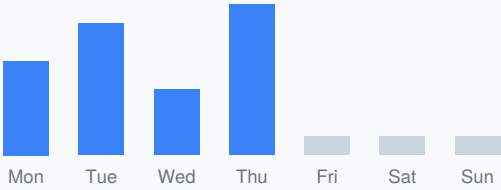


Status:
Clocked Out

Worked today:
0h 0m

THIS WEEK

Total so far: **18.5h**



Upcoming Shifts

Wed, Aug 26 · 9:00 AM – 5:00 PM Downtown Store	PENDING	Accept	Reject
Fri, Aug 28 · 12:00 PM – 8:00 PM Westfield Kiosk	ACCEPTED		
Sat, Aug 29 · 9:00 AM – 3:00 PM Downtown Store	ACCEPTED		

Full history and hours-worked detail live on the Timesheet tab.

Clock in/out, today's and this week's hours, and upcoming shifts with accept/reject actions.



My Timesheet

Week of Aug 18–24 ▼

Total hours: **32.5h** Days worked: **5** Shifts completed: **5**

DATE	CLOCK IN	CLOCK OUT	HOURS	LOCATION
Mon, Aug 18	8:02 AM	4:00 PM	7.97h	Downtown Store
Tue, Aug 19	9:00 AM	5:05 PM	8.08h	Downtown Store
Wed, Aug 20	—	—	0h	Not scheduled
Thu, Aug 21	12:00 PM	8:00 PM	8.0h	Westfield Kiosk
Fri, Aug 22	9:00 AM	3:30 PM	6.5h	Downtown Store
TOTAL			32.5h	

Hours are calculated automatically from clock-in / clock-out timestamps.

Full history: clock in/out per day, hours worked, and weekly totals.



Shift Staffing

◀ Week of Aug 24 – Aug 30 ▶

Every shift needs a minimum of 3 and a maximum of 5 staff.

DATE / TIME	LOCATION	STAFFED	STATUS	ACTION
Tue Aug 26 9:00 AM – 5:00 PM	Downtown Store	2 / 5 	NEEDS STAFF	<button>Manage</button>
Tue Aug 26 5:00 PM – 11:00 PM	Downtown Store	3 / 5 	PENDING ACCEPT	<button>View</button>
Wed Aug 27 9:00 AM – 5:00 PM	Westfield Kiosk	5 / 5 	FULLY STAFFED	<button>View</button>
Thu Aug 28 12:00 PM – 8:00 PM	Downtown Store	0 / 5 	UNFILLED	<button>Manage</button>

Clicking "Manage" on a shift opens the assign-employees panel (see next wireframe).

Every shift for the week with fill status (Needs Staff / Pending Acceptance / Fully Staffed / Unfilled).

Assign Employees

×

Tue, Aug 26 · 9:00 AM – 5:00 PM

Downtown Store

2 of 5 assigned · min 3 required

EMPLOYEES AVAILABLE FOR THIS SLOT

Alicia Munoz

Available 8:00 AM – 6:00 PM

ASSIGNED ✓

Devon Park

Available 7:00 AM – 4:00 PM

PENDING

Priya Shah

Available 9:00 AM – 5:00 PM

Assign

Marcus Lee

Available 8:30 AM – 4:30 PM

Assign

Jordan Reyes

Declined this shift

DECLINED

Declining auto-offers the shift to the next available employee in the list.

Done

Opens when a manager clicks a shift: everyone available for that slot, with assign / pending / declined states.

ShiftApp

MANAGER

Weekly Rost

CONFIRMED · AUG 24–30

Export

	MON 24	TUE 25	WED 26	THU 27	FRI 28	SAT 29	SUN 30
Morning 7am – 3pm	Alicia M. Devon P. Priya S.	Alicia M. Marcus L. Priya S.	Devon P. Marcus L. Jordan R.	UNFILLED 0 of min 3	Alicia M. Priya S. Marcus L.	Devon P. Jordan R. Priya S.	No shift
Evening 3pm – 11pm	(same chip pattern continues for the evening shift row)						

A day only shows here once every assigned employee for that shift has accepted, and shows red if the minimum of 3 wasn't reached.

The confirmed roster: days × shift times with assigned employees as chips, unfilled days flagged in red.

8. Suggested Tech Approach (optional starting point)

Your developer will have their own preferences, but as a reasonable default for this scope:

- **Frontend:** React (or similar SPA framework), calendar/table components for availability and roster grids.
- **Backend:** Node.js/Express, Python/Django, or Rails — anything with solid background-job support, since auto-reassignment and roster confirmation are best run as async jobs rather than blocking requests.
- **Database:** PostgreSQL — relational fits this data model well (shifts, assignments, time entries all reference each other).
- **Auth:** JWT-based sessions, role field on the user for employee vs. manager.
- **Notifications:** start with in-app only for MVP; add email (and SMS if needed) once the core flow is proven.

9. Open Questions to Settle Before Build

A few decisions the original brief doesn't cover yet — worth resolving with your developer early, since they affect the data model:

1. **Notification channel** for shift offers, rejections, and reassignment — in-app only, email, SMS, or push?
2. **Response deadline** — how long does an employee have to accept/reject an offer before it auto-expires and moves to the next person?
3. **Reassignment order** — when multiple employees are available for a slot, who gets offered first? (First to submit availability, seniority, fewest hours this week, manager-set priority, etc.)
4. **Unfilled shifts** — if the pool runs out before hitting the minimum of 3, what should the manager be able to do? (Broadcast an open call-out, manually add someone outside their stated availability, adjust shift times?)
5. **Editing availability** — can an employee change availability after submitting, and does that retroactively affect shifts they're already offered/assigned to?
6. **Multiple locations** — is this single-location for now, or do managers/employees operate across several sites from day one?
7. **Shift creation** — who defines the shifts themselves (date/time/location) that employees are asked to be available for — is that manual, from a recurring template, or auto-generated?
8. **Shift swapping** — once accepted, can two employees trade shifts directly, or does everything route through the manager?
9. **Time zone handling** — relevant if locations span multiple time zones.
10. **Overtime / breaks** — does the time clock need to track unpaid breaks or flag overtime thresholds?

None of these block starting the build — they're mostly MVP-vs-later calls — but pinning them down now avoids rework.

10. Suggested Build Phases

Phase 1 — MVP

- Auth (employee/manager roles)
- Availability entry (rolling 14-day window)
- Manager staffing board + manual assignment
- Accept/reject with auto-reassignment
- Clock in/out + basic daily/weekly hours
- Weekly roster view

Phase 2 — Enhancements

- Email/SMS notifications
- Shift swapping between employees
- Multi-location support
- Recurring shift templates
- Reporting/exports (payroll-ready hours export, staffing analytics)
- Mobile app or PWA polish